Cheat sheet

Python

Syntax, data structures, OOP, idioms and interview gotchas

Basics

Types	`int`, `float`, `str`, `bool`, `None`, `list`, `tuple`, `dict`, `set`
Mutable	list, dict, set (and most objects). Immutable: int, float, str, tuple, frozenset
Truthy / falsy	Falsy: `0`, `""`, `[]`, `{}`, `None`, `False`. Everything else is truthy
Identity vs equality	`is` compares objects, `==` compares values. Use `is None`
f-string	`f"{name!r} {price:.2f} {pct:.1%}"`
Slicing	`s[start:stop:step]`, `s[`
Unpacking	`a, *rest = [1, 2, 3]`, `x, y = y, x` swaps

Data structures

python

```python
nums = [3, 1, 2]; nums.append(4); nums.sort(); nums.pop()       # list
d = {"a": 1}; d.get("b", 0); d.setdefault("c", []); d.items()   # dict
s = {1, 2} | {2, 3}; s & {2}; s - {1}                           # set ops
from collections import Counter, defaultdict, deque
Counter("banana").most_common(1)      # [('a', 3)]
g = defaultdict(list); g["k"].append(1)
q = deque([1, 2]); q.appendleft(0); q.popleft()
import heapq; heapq.nsmallest(2, [5, 1, 4])
```

Comprehensions and functional

List	`[x*x for x in xs if x > 0]`
Dict	`{k: v for k, v in pairs}`
Generator	`sum(x*x for x in xs)` (lazy, memory-friendly)

Sorting	`sorted(items, key=lambda r: (-r["score"], r["name"]))`
zip / enumerate	`for i, (a, b) in enumerate(zip(xs, ys))`
any / all	`any(x < 0 for x in xs)`

Functions

Defaults	Never use mutable defaults: `def f(x, acc=None): acc = acc or []`
***args / **kwargs**	Variable positional / keyword arguments
Keyword-only	`def f(a, *, verbose=False)`
Closures	Inner function remembers outer variables (`nonlocal` to modify)
Decorator	`@functools.wraps` keeps name and docstring
Type hints	`def f(x: list[int]) → dict[str, int]:`

OOP

python

```python
from dataclasses import dataclass
class Animal:
    count = 0                        # class attribute
    def __init__(self, name): self.name = name; Animal.count += 1
    def speak(self): raise NotImplementedError
    def __repr__(self): return f"Animal({self.name!r})"
class Dog(Animal):
    def speak(self): return "Woof"
@dataclass(frozen=True)
class Point: x: float; y: float
```

Dunder methods	`__init__` , `__repr__` , `__eq__` , `__len__` , `__iter__` , `__enter__/__exit__`
@staticmethod vs @classmethod	No self / receives the class (`cls`), used for alternative constructors
@property	Method accessed like an attribute, with validation

MRO	Method resolution order for multiple inheritance (`Class.__mro__`)

Errors, files, JSON

```python
try:
    data = json.loads(text)
except json.JSONDecodeError as e:
    log.warning("bad json: %s", e)
else:
    process(data)
finally:
    cleanup()
with open("f.txt", encoding="utf-8") as f:
    for line in f: ...
```

Concurrency

GIL	One thread runs Python bytecode at a time in standard CPython
threading	I/O-bound work (API calls, downloads)
multiprocessing	CPU-bound work
asyncio	Many concurrent I/O tasks: `await asyncio.gather(*tasks)`

Complexity

list	index O(1), append O(1), insert / search / remove O(n)
dict / set	get, set, in: O(1) average
sort	O(n log n) (Timsort, stable)

> ⚠ **Watch out**
>
> Classic traps: mutable default arguments, modifying a list while iterating, `is` vs `==`, integer division `//`, late binding in lambdas inside loops, shallow copies of nested lists.